What Engineering True Cost Is

Why Software Can't be Credentialed and the Solution

Registry: [HOWL-ENG-2-2026]

Series Path: [HOWL-ENG-1-2026] → [HOWL-ENG-2-2026]

DOI: 10.5281/zenodo.19997507

Date: May 3 2026

Domain: Applied Philosophy

Status: Working Methodology

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.7.

---

1. Why this paper exists

The first paper defined engineering. It identified True Cost as the qualifier that distinguishes engineering from academic work — the property that failure imposes harm on parties outside the engineering activity itself. The treatment of True Cost in that paper was binary. Either the activity has it, in which case the structural criterion can be met, or it doesn't, in which case the activity is academic regardless of how rigorous it is.

The binary treatment was sufficient to draw the engineering boundary. It is not sufficient to answer a question the software industry has been failing to resolve for fifty years: should software engineers be credentialed, and if so, how?

Attempts have been made. The IEEE Computer Society offers software engineering certifications. ABET accredits software engineering degree programs. Some jurisdictions have offered software engineering PE exams. Various national and international bodies have proposed licensure schemes. None of them have produced credentials the industry takes seriously, none have produced the kind of professional accountability structure that civil or electrical engineering has, and none have answered the basic question of who should certify what when software fails.

Meanwhile, software increasingly controls systems whose failure kills people. Autonomous vehicles must decide between collision outcomes. Medical devices deliver treatment in ways where bugs end lives. Industrial control systems manage chemical processes whose failure can level neighborhoods. Avionics software flies aircraft. Surgical robots cut into bodies. Grid management software dispatches power to hospitals on backup. The regulatory regimes around these domains have evolved without ever clearly explaining why they certify what they certify. The software industry remains confused about whether its own engineers should be credentialed, by whom, and for what.

This paper resolves the confusion. The resolution comes from unfolding True Cost into its domains. Once the domains are visible, the credentialing question answers itself.

The paper is focused on software engineering. Operations work is mentioned for completeness because the structural argument applies there too, but the central concern is SWE: what is the True Cost of typical software work, what is the True Cost of software work that crosses into mortality-bearing domains, and what credentialing makes sense in each case.

2. True Cost is not one thing

The first paper said True Cost is harm borne by goal-seekers and users when the engineered system fails. That definition is correct. It is also undifferentiated. The harm can land in different domains, and the domains have structurally different properties.

Two domains in particular are worth naming clearly.

Physical and civilizational True Cost. The cost lands as physical harm or civilizational disruption. People die directly: a bridge collapses and kills drivers, a building burns and kills occupants, a pressure vessel ruptures and kills workers, a medical device delivers a fatal dose, an aircraft falls. People die indirectly: power fails and hospitals lose backup, water treatment fails and a city gets sick, transportation fails and food doesn't move, communications fail and emergency services can't dispatch. Civilization itself is disrupted: trains stop running, people can't get to work, goods don't move, supply chains fail, the basic conditions that make modern life possible are interrupted.

The cost is largely irreversible. Dead people stay dead. A starvation event leaves permanent damage to a population. A city without power for a week has accumulated harm that can't be undone retroactively. The asymmetry between the engineering decision (made in minutes, by one person) and the cost (paid in lives, across populations, across time) is severe and structural.

Financial and service-availability True Cost. The cost lands as financial harm to organizations and degradation of services to users. Companies die: Friendster, MySpace, Digg, hundreds of startups whose technical or operational failures combined with competitive pressure produced extinction. Services degrade: outages cost revenue per minute, slow pages drive users to competitors, broken flows damage trust, missing images embarrass the brand. Data loss can be catastrophic for affected users (medical records, financial records, personal communications), though usually it's recoverable from backups or by re-creating state. Outages, frustration, defection, opportunity cost.

The cost is largely reversible. Businesses can recover from outages. Users can be reacquired. Data can be restored. Trust can be rebuilt over time. The reversibility is partial — a business that dies stays dead, a user who left is hard to win back, data lost without backup is gone — but the failure modes don't have the irreversibility floor that physical engineering's failures have. Nobody dies because your req/s went up.

These are different kinds of True Cost. Both are real. Both qualify under the engineering definition. They differ in magnitude, in reversibility, and in the credentialing infrastruc-

ture that makes sense around them. The remainder of this paper works through what follows from this difference.

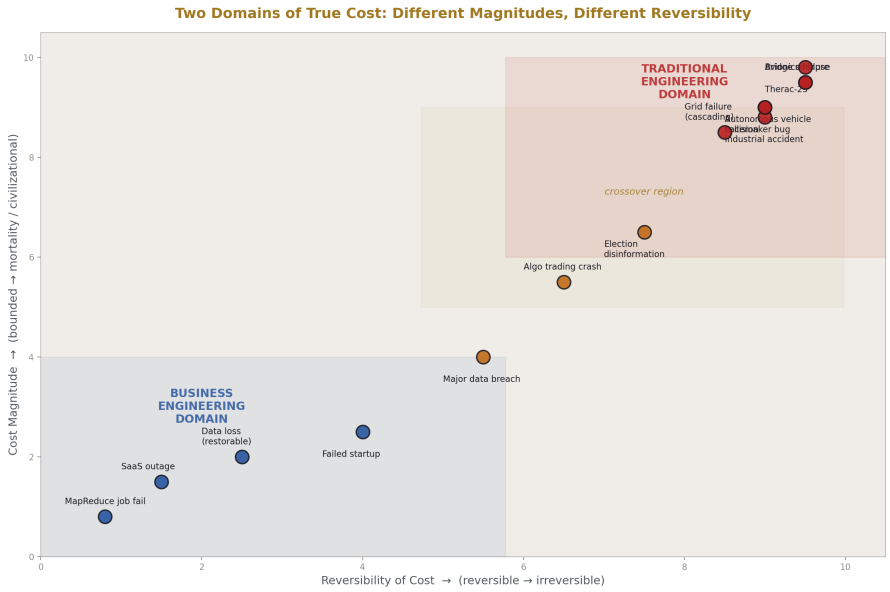

Figure 1: Fig. 1: Two Domains of True Cost — physical/civilizational costs are mortality-bearing and irreversible; financial/availability costs are bounded and reversible.

3. Why traditional engineering can be credentialed

Traditional engineering means civil, mechanical, electrical, structural, chemical, aerospace, biomedical, and the other disciplines that operate against physical reality and produce physical or civilizational True Cost. Every one of these disciplines has formal credentialing — the PE in the United States, the Chartered Engineer process in the UK and Commonwealth, the ingénieur diplômé in France, equivalents across the world. The credentialing works. It produces practitioners whose competence is reasonably reliable, whose work is signed and stamped, whose accountability is anchored by a license that can be revoked.

It works because the substrate holds still.

Concrete cures the way it cured fifty years ago. Steel has the yield strength it had a century ago. Gravity is the same. Soil mechanics is the same. Wind loads are calculated against fluid dynamics that hasn't changed. Thermal expansion coefficients of materials are constants in the strict sense. Electrical permittivity, magnetic permeability, the speed of light in vacuum — these are physical constants that don't drift. Building codes evolve slowly because the underlying physics evolves not at all. New materials enter the field over decades; new analysis methods arrive over decades; the body of practice accumulates

incrementally and what an engineer learned in 1995 is mostly still applicable in 2026.

This stability allows three things that credentialing requires.

A **stable body of knowledge** that can be tested. The PE exam is a serious examination of a serious body of knowledge that doesn't change much from decade to decade. The exam in 1995 tests largely the same competence as the exam in 2026 because the underlying engineering hasn't changed. New chapters get added to codes, new analysis methods get incorporated, but the foundation is intact.

Portable competence. A licensed structural engineer can design a bridge in California and a different bridge in Texas using the same skills. The competence transfers because the substrate is shared. The same engineer can move between projects, between firms, between specialties within their discipline, and the credentialed competence remains relevant. The license certifies a body of skill that is genuinely the engineer's, not tied to a specific project or employer.

Accumulated formal codes. IBC, ACI, AISC, ASCE 7, NEC, IEC, ASHRAE, ASME, and dozens of others. These are written standards that codify the body of knowledge. Engineers are tested against them. Outputs are evaluated against them. Failures are investigated against them. The codes are the formal anchor that allows credentialing to be more than reputation — they give the credential a substrate-grounded meaning.

The combination of stable substrate, portable competence, and formal codes is what makes person-level credentialing work. Society credentials the person; the person stamps outputs; the chain of accountability runs through the person and is anchored by the credential.

4. Why traditional engineering must be credentialed

The cost-magnitude argument. Society credentials professions when the cost magnitude justifies the overhead of credentialing. Bridges fall and people die. Buildings burn and people die. Pressure vessels rupture and people die. Power systems fail and hospitals lose their backup, surgeries get interrupted, life-support equipment fails. Water treatment fails and cities get sick. Transportation fails and food doesn't move. The mortality and civilizational disruption of traditional engineering failures is severe enough that society absorbs the cost of building the credentialing apparatus.

That apparatus is expensive. Licensure boards. Examination bodies. Continuing education requirements. Professional liability frameworks. Disciplinary boards that can revoke credentials. Code-development organizations. Peer review processes. Insurance regimes built around licensed practice. Legal frameworks that assign liability to the licensed engineer when work fails.

Society pays this cost because the alternative is worse. Without credentialing, the asymmetric, irreversible nature of physical and civilizational True Cost falls on the public without any mechanism to anchor accountability. With credentialing, a licensed engineer signing a stamp is taking on personal liability that can ruin them if their work fails. The credential creates skin in the game where otherwise there would be none. The public

knows that someone competent has reviewed the work and is accountable for it. The system isn't perfect — engineers fail, codes are insufficient, accidents happen — but the alternative of unlicensed practice in mortality-bearing fields is recognized everywhere as worse.

This is the case for credentialing. It rests on cost magnitude, substrate stability, and the social calculation that the overhead of credentialing is justified by what it prevents. All three conditions hold for traditional engineering. They do not hold for software in general.

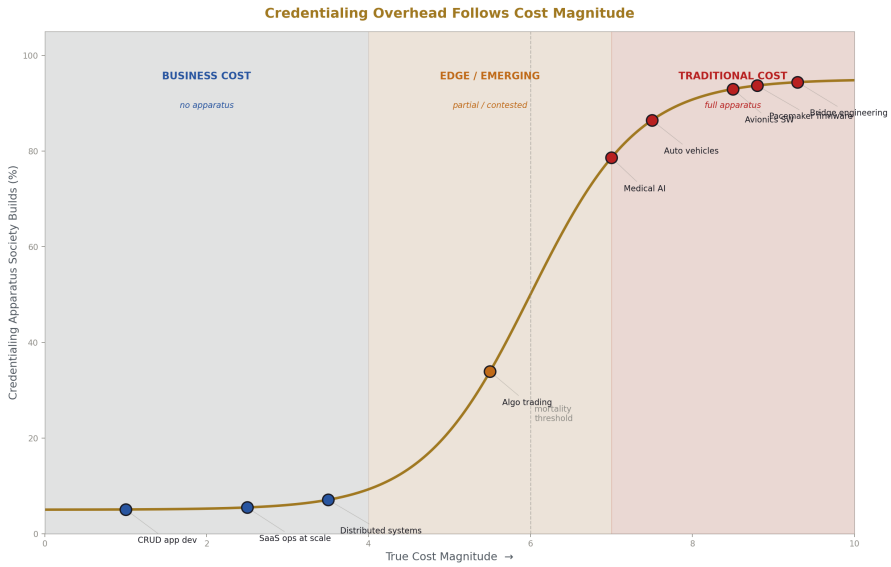

Figure 2: Fig. 4: Society's credentialing apparatus scales with cost magnitude — full at mortality, none at bounded financial cost.

5. Why software engineering cannot be credentialed at the person level

The substrate-instability argument. Software engineering operates against a substrate that is itself engineered, by other software engineers, who keep changing it. There is no underlying stable physical reality the way there is for civil or electrical engineering. The substrate is whatever the current operating systems, network protocols, language runtimes, frameworks, libraries, cloud APIs, and tools happen to be at this moment, and all of those move continuously.

Consider what a software engineer in 1999 was working with: Solaris, Linux 2.2, Windows NT, Perl, C++, Java 1.2, PHP 4, mod_perl, Apache 1.3, MySQL 3.23, MS SQL Server 7, raw sockets, CGI, IRC for team comms, mailing lists, manual deploys, FTP, Slackware. A software engineer in 2010 was working with Linux 2.6, Solaris dying, Windows Server 2008, Python 2.6 and Ruby on Rails, Java 6, PHP 5.3, Nginx, MySQL 5,

PostgreSQL 9, EC2, S3, Memcached, jQuery, IRC and Skype, Subversion fading to Git, REST APIs, JSON. A software engineer in 2026 is working with Linux 6.x, Windows Server still around, Rust and Go and TypeScript and Python 3.13, Kubernetes, AWS and GCP and Azure, Terraform, gRPC, Kafka, Slack, Git, GitHub Actions, language models in the toolchain, ephemeral containers, serverless functions.

These are different stacks solving different problems with different tools. The 1999 engineer's expertise in Perl mod_perl and Apache module development is essentially worthless in 2026. The 2010 engineer's expertise in Rails 3 plugins and EC2 classic networking is largely obsolete. The 2026 engineer's expertise in current Kubernetes operators and current AWS service catalogs will be substantially obsolete by 2035.

This isn't an unusual amount of churn. This is the normal pace of software substrate change. Every few years, large pieces of the practice are replaced. Frameworks come in waves. Languages rise and fall. Architectural patterns cycle. The substrate that the work operates against is engineered, and the engineering of the substrate keeps getting redone, sometimes for good reasons, sometimes for bad ones, but always.

Three problems for credentialing follow from this directly.

No stable body of knowledge to test against. A credential earned against the 1999 substrate would be substantially obsolete by 2010 and almost completely obsolete by 2026. Whatever a software engineering credential tested for in 1999, the tested knowledge would not predict competence in 2010 or 2026. This isn't a flaw in the credential design. It's a structural fact about what software engineering operates against. The body of knowledge isn't stable enough to anchor a credential.

Some attempts to credential have tried to solve this by testing against fundamentals — algorithms, data structures, software design principles, computer organization. These fundamentals are stable. They're also nearly useless as a measure of professional competence at building current systems. Knowing big-O notation and being able to write a balanced binary tree doesn't predict whether someone can build a reliable distributed system on current cloud infrastructure. The fundamentals are necessary but nowhere near sufficient. A credential that tests only the stable parts is testing for a small slice of the relevant competence.

Other attempts have tried to credential against specific stacks — AWS Solutions Architect, Kubernetes Certified Administrator, various vendor certifications. These are often useful as job-market signals but they go obsolete on a faster timescale than the credentials they're supposed to be analogous to. Today's AWS certification is partially obsolete in three years. The PE doesn't have this problem because steel doesn't deprecate. Kubernetes does.

Non-portable competence. The same engineer in the same role at the same company in 1999, 2010, and 2026 was doing different work. Competence at one moment doesn't transfer cleanly to another moment within the same person's career. Certainly competence doesn't transfer across people in the way civil engineering competence does. Two software engineers with the same number of years of experience often have entirely non-overlapping skill sets, because they happened to work on different parts of the substrate during different time periods.

This is why hiring in software relies so heavily on interviews testing current ability rather than on credentials. The market has figured out that credentials don't predict performance well, because the body of knowledge they would have tested for has moved by the time the credential is being relied on.

No formal codes. Civil engineering has IBC, ACI, AISC, ASCE 7. Electrical engineering has NEC, IEC. Mechanical engineering has ASME. Software engineering has…books, conference talks, blog posts, tribal knowledge, and a fragmented landscape of style guides and best-practice documents that often contradict each other and go out of fashion within a decade. There is no formal standard analogous to a building code. There is no equivalent of "the wiring in this building must conform to NEC 2023" for "the architecture of this software system must conform to ANSI/IEEE-XXXX-2026." It doesn't exist. It can't exist in the same form because the substrate is too fluid.

These three problems compound. A credential needs something stable to test against, and software doesn't provide it. A credential needs to predict portable competence, and software competence isn't portable in the way traditional engineering competence is. A credential needs formal codes to anchor it, and software lacks them. Every attempt to build software credentialing runs into these structural facts. The credentials that result are weak — they don't predict performance, they don't produce accountability, they don't carry the weight of a PE or a Chartered Engineer designation. This is not a failure of the credential designers. It is the substrate refusing to support the credentialing model that traditional engineering uses.

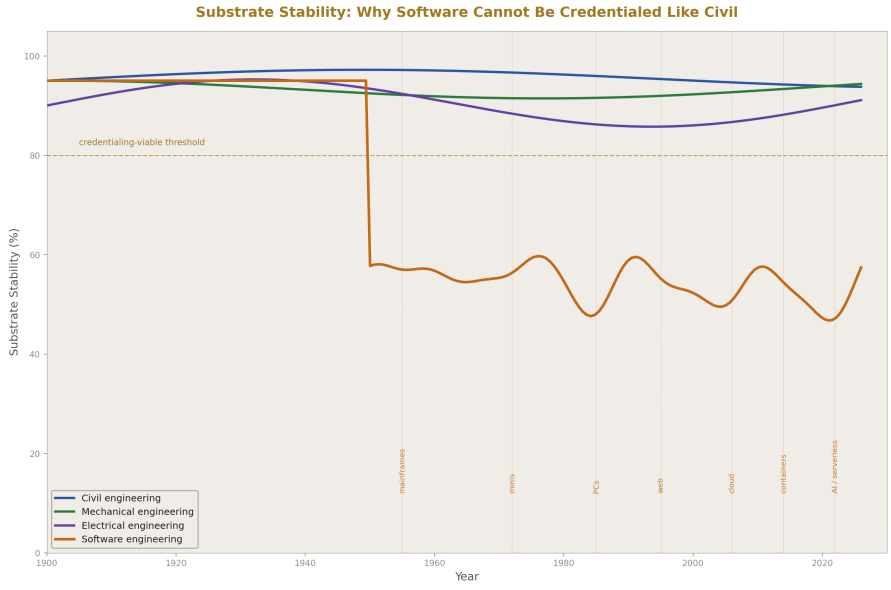

Figure 3: Fig. 2: Substrate stability over time — civil/mechanical/electrical operate on stable substrates; software's substrate churns continuously.

6. Why software engineering doesn't need credentialing

The cost-magnitude argument. The other half of the case. Even if software credentialing could be made to work technically, society doesn't need it for most software work because the costs don't justify the overhead.

Most software engineering work is in the financial and service-availability True Cost domain. A failed startup is an economic loss. A degraded SaaS platform is an inconvenience. A buggy mobile app annoys users. A slow website costs page views. A broken e-commerce flow loses sales. These costs are real. They are also bounded, mostly reversible, and not mortality-bearing. Society has not built credentialing infrastructure around any of these because the cost magnitudes don't warrant the apparatus.

The market handles competence assessment for software through other mechanisms. Reputation. Interview performance. Code samples. Portfolios. Track records. References. Employment history. GitHub contributions. Open source involvement. Trial periods. Probationary employment. Performance reviews. Firing. The mechanisms are messy and unfair in many ways, but they're calibrated to the cost magnitudes involved. A bad software engineer at a typical SaaS company can be identified, can be replaced, and the cost of having had them is bounded. Society doesn't need a license to handle this.

A useful comparison: society credentials physicians extensively because medical errors kill people. Society credentials lawyers extensively because legal errors imprison or impoverish people. Society credentials civil engineers extensively because structural failures kill people. Society does not credential web developers because failed websites are an economic problem, not a mortality problem. The pattern is consistent: credentialing follows mortality and severe civilizational cost. Where the cost is bounded and reversible, credentialing isn't built.

This isn't a deficiency in the software field's professional development. It's an accurate match between the field's actual cost structure and the social infrastructure that makes sense around it. Software engineers are not denied credentials because the field is immature. They are not credentialed because the costs of their typical work don't justify the credentialing overhead, and the substrate they work against won't support the credentialing model anyway.

7. Crossover engineering: where software touches traditional cost domains

Most software engineering is in the business engineering tier and stays there. Some software engineering crosses into traditional-engineering cost territory. The two cases are structurally different and require different responses.

Crossover engineering means software work whose failure modes are physical, civilizational, or otherwise at traditional-engineering cost magnitudes. The clearest examples:

- **Autonomous vehicles** that must decide between collision outcomes — when a crash is unavoidable, the software is choosing who gets hit and how. Failure mode is mortality.

- **Medical devices** that deliver treatment — pacemakers, infusion pumps, insulin pumps, implantable defibrillators, surgical robots, radiation therapy controllers. Failure mode is patient harm or death.
- **Avionics software** that controls flight systems, navigation, autopilot, fly-by-wire. Failure mode is aircraft loss with passengers and crew.
- **Industrial control systems** managing chemical processes, refineries, power generation, water treatment. Failure mode is industrial accidents that can kill workers and surrounding populations.
- **Grid management software** dispatching power, balancing load, managing failover. Failure mode is blackouts that cascade into hospital backup failures, traffic system failures, and broader civilizational disruption.
- **Industrial process control** for manufacturing systems whose failure can level facilities or release toxic materials.

In all of these, software is doing trade-off evaluation against externalities where failure has traditional-engineering True Cost. The structural criterion from the first paper is met — substrate, alignment, externalities, cost — and the cost magnitude is at the traditional tier rather than the business tier. Mortality is on the line. Civilizational disruption is on the line. The reversibility floor of business engineering doesn't apply.

This is real engineering at real magnitude. The engineers building these systems are doing engineering in the strict sense, and their work imposes real consequences on real third parties. The credentialing question follows the cost: where the cost is at the traditional tier, credentialing should apply.

But the credentialing target cannot be the software engineer as a person. Three structural reasons.

Software competence is not portable across crossover domains. A software engineer who has worked on automotive collision avoidance is not, by virtue of that experience, qualified to work on heart pacemaker firmware. The substrates are different, the failure modes are different, the regulatory contexts are different, the timing constraints are different, the validation methods are different. The body of domain knowledge required to know what counts as "correct decisions" varies enormously across these fields. Two crossover-tier domains share less in common than two civil engineering specialties share, because the underlying substrate of each is the specific engineered substrate of the specific product, not a shared physical reality.

A traditional engineering analogy. A licensed structural engineer can design a bridge in California and a building in Texas using overlapping competence, because both work against gravity, soil, wind, and the same materials. A software engineer who has built pacemaker firmware does not, by virtue of that experience, have the relevant competence to build avionics software. The bodies of domain knowledge are largely disjoint. Person-level credentialing in software for crossover work would either certify someone for one specific domain (in which case the credential is so narrow it barely justifies the apparatus) or certify them generally (in which case the credential overstates what they actually know).

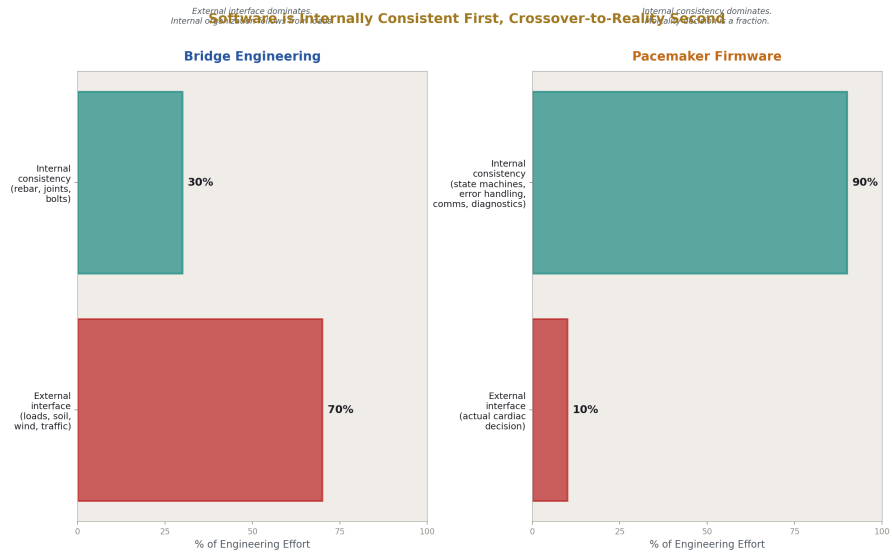

Figure 4: Fig. 5: Engineering effort distribution — bridge work is dominated by external interface; pacemaker firmware is dominated by internal consistency.

Software is internally consistent first and crossover-to-reality second. This is the deepest structural reason and worth slowing down on.

A bridge's engineering is fundamentally about its relationship with physical reality. The internal structure of the bridge — the rebar layout, the cable arrangement, the bolt patterns — exists in service of the external interface with reality (loads, wind, soil). The internal consistency of the design is downstream of the external relationship. A bridge that is internally consistent but doesn't match its loads is wrong; a bridge that matches its loads is right, and the internal organization follows from that.

Software is the inverse. The internal consistency of the software is the primary engineering concern, and the relationship with reality is one of many things the software does, often a small part. A pacemaker's firmware spends most of its complexity on internal logic — state machines, error handling, communication protocols, configuration management, diagnostic functions, telemetry, update mechanisms, watchdog timers, fault tolerance, redundancy management. Only a small fraction of the codebase touches the actual cardiac pacing decision that matters for mortality. The internal consistency dominates the external interface in terms of where the engineering effort goes.

Because software is internally consistent first, the engineer's competence at making *that specific system* internally consistent is what determines whether the output is correct. The competence isn't transferable to a different system, because the different system has different internal consistency requirements that have nothing to do with the engineer's prior work. Certifying the person is certifying something that doesn't predict their next system's

correctness. Certifying the output is the only meaningful claim.

A thousand civil engineers designing the same bridge for the same site converge on similar broad shapes because the underlying physics constrains the solution space heavily. The bridge that emerges from one engineer is recognizably similar to the bridge that emerges from another. The competence of any one of them is portable to the next bridge because the constraints are portable.

A thousand software engineers designing the same crossover system would produce a thousand wildly different architectures, with different state representations, different error handling philosophies, different module decompositions, different testing approaches, different concurrency models, different update strategies. The variation isn't a sign of immaturity or lack of discipline. It's a structural feature of software as a substrate. Internal consistency demands choices that don't generalize, and the choice space is enormous. Two competent software engineers presented with the same problem will produce systems that differ in ways that matter for correctness, because their internal-consistency choices were different from the start.

The certification has to attach to the system, not to the person. The system is the unit that has internal consistency. The system is what crosses to reality at specific points. The system is what makes specific decisions in specific contexts. The certification says "this system, validated for this use, has been examined and found to make correct decisions within its design envelope." It doesn't say "the person who wrote it is competent to write the next one," because that claim isn't structurally true for software.

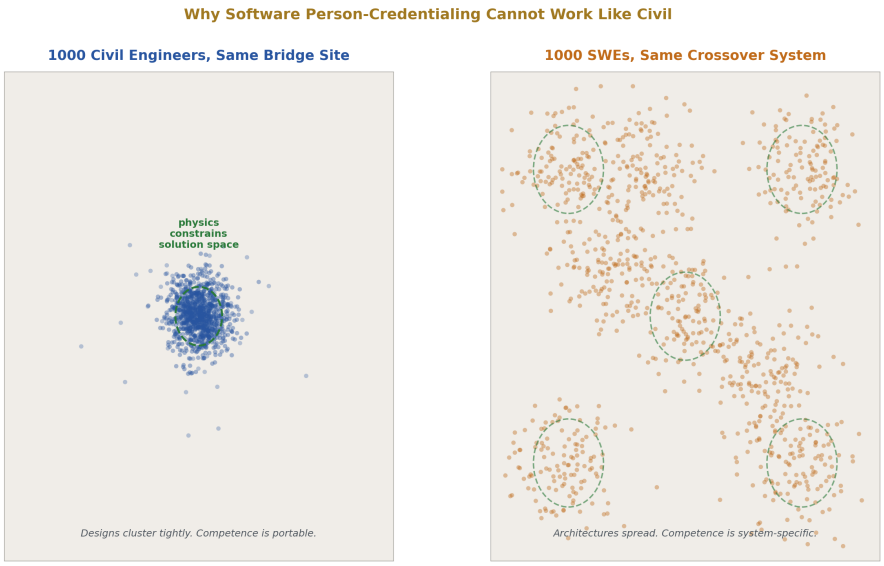

Figure 5: Fig. 6: Convergence vs divergence — civil engineers cluster around physically constrained solutions; SWEs spread across the architectural space.

8. Who certifies crossover engineering

The certifier must be a domain-credentialed traditional engineer, not a software engineer.

The crossover certification asks: does this system make correct decisions when it touches reality at this point? Answering requires knowing what "correct" means in the domain. Correct in pacemaker territory means correct cardiac pacing under the full range of physiological conditions a pacemaker will encounter — across patient populations, across cardiac conditions, across electrolyte states, across drug interactions. Correct in autonomous vehicle decisions means correct collision-avoidance behavior across the full range of road and traffic conditions, weather, lighting, sensor degradation, pedestrian populations. Correct in industrial control means correct process management across the full range of input variations, equipment states, and failure conditions.

The software engineer who built the system can verify internal consistency. They can run tests, formal methods, simulations, fuzzing, property-based testing. They can verify that the code does what they meant it to do. What they cannot reliably verify is whether what they meant it to do is correct in the domain. That requires domain expertise — knowing what the human heart actually needs from a pacemaker, what physical dynamics govern a vehicle's stopping distance on wet pavement at night with a degraded camera, what process upset modes a chemical plant exhibits and which interventions are safe.

Domain expertise is the body of knowledge that traditional engineering accumulates and credentials. The biomedical engineer or cardiac physiologist who has built and verified medical devices for decades knows what correct pacemaker behavior looks like across the patient population. The aerospace engineer with avionics experience knows what correct autopilot behavior is across flight regimes. The chemical engineer with process control experience knows what safe operation looks like for their plant type. Their PE or equivalent credential is the formal recognition that they hold the body of knowledge needed to evaluate correctness in their domain.

When software crosses into their domain and starts making decisions that touch their substrate, they're the right people to sign off on whether the decisions are correct. They are not certifying the software's internal construction; they are certifying that the system's external behavior, in the operating envelope it's approved for, makes the right calls in their domain. The software engineer made it work. The traditional engineer says it works correctly.

This also explains why the certifier should not be a software engineer. A software engineer certifying crossover software is being asked to certify something outside their competence. They can certify that the code is well-constructed, well-tested, internally consistent. They cannot certify that the decisions the code makes are correct in the medical, chemical, civil, or whatever domain the code touches. The competence to make that claim lives in the traditional engineering discipline whose substrate the software is operating on.

If you tried to credential software engineers to do crossover certification, you'd have to teach them medicine, chemistry, traffic engineering, or whatever domain applies. At that point you'd have built a worse version of the existing traditional engineering credential by adding software construction skills to a domain expert. It's more efficient to take

the existing domain expert and give them enough software systems literacy to evaluate the system's external behavior than to take a software engineer and teach them domain expertise from scratch.

This is also what the existing regulatory regimes have done in practice, often without articulating why. **DO-178C** in avionics certifies the software development process, the verification artifacts, and the resulting code. The reviewers are aerospace engineers and human-factors specialists, with software engineers contributing to the technical review but not making the final airworthiness call. **FDA medical device regulation** uses the 510(k) and PMA processes, which certify the device — including its software — for specific indications. The reviewers are physicians and biomedical engineers. **ISO 26262** for automotive functional safety certifies the system and the development process, with safety engineers credentialed in functional safety and automotive systems making the final call. **IEC 61508** plays the same role across general industrial functional safety.

The pattern is consistent: domain experts certify, with software expertise contributing. These regulatory regimes have already converged on the output-certification model for crossover work, and they have already converged on domain-engineer-as-certifier. The framework presented here names what they discovered.

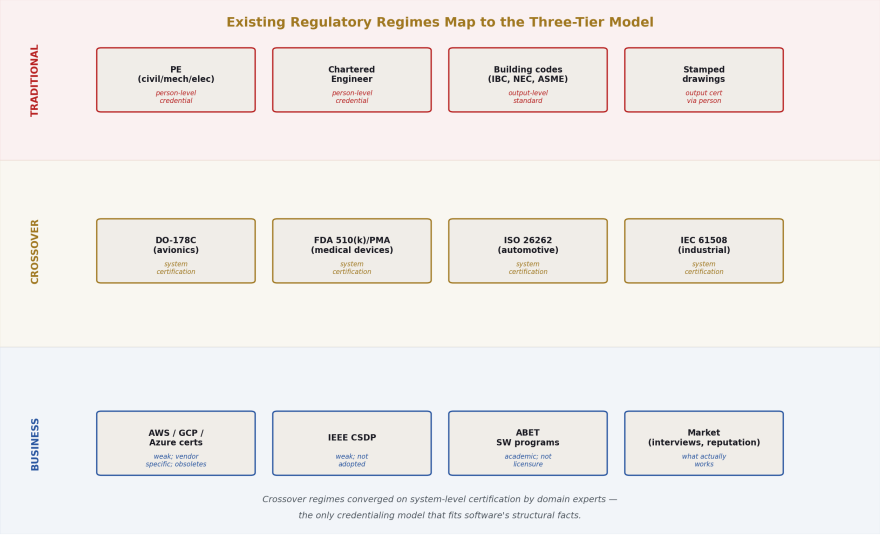

Figure 6: Fig. 8: Existing regimes mapped to the framework — DO-178C, FDA, ISO 26262, IEC 61508 all converged on system-level certification by domain experts.

9. The two-role model for crossover work

Crossover engineering staffs two distinct engineering roles. Both are doing real engineering by the structural definition. Neither alone is sufficient. Together they produce the

certified output.

The **software engineer** builds the system. This is real engineering: they evaluate trade-offs in software construction against externalities (the system's behavior, its performance, its failure modes, the operating environment, the validation requirements) with True Cost on whether the system works. They produce an internally consistent artifact, validated against requirements, tested rigorously, documented thoroughly, maintainable across its lifecycle. The work is engineering at crossover-tier cost magnitudes. They do not need an individual credential because their competence is in software construction, which isn't portable in a way that credentialing supports.

The **domain engineer** certifies the system. This is also real engineering: they evaluate trade-offs in domain risk against externalities (the substrate the system touches — the patient population, the road conditions, the process dynamics, the airframe — and the failure modes that can occur at the software-substrate boundary) with True Cost on whether the certification is sound. They produce a certification artifact that says: this system, validated under these conditions, in this operating envelope, behaves correctly in our domain. Their PE or equivalent credential is the formal anchor of the certification. Their license is on the line if the certification is wrong.

The two roles are different engineering activities and they require different bodies of knowledge. The software engineer's knowledge is internal-consistency knowledge — language, framework, system design, validation methods, testing approaches, debugging, deployment, maintenance. The domain engineer's knowledge is substrate knowledge — the physics, biology, chemistry, or physiology of the domain, the failure modes that can occur in reality, the operating conditions the system will face, the codes and standards that govern the domain.

The accountability chain runs through the certifier. When a certified system fails in production and causes harm, the certifier (the domain-credentialed engineer who signed off) and the manufacturer (whose process produced the system) are formally accountable. The software engineer may face professional consequences — they may be involved in the investigation, their employment may end, their professional reputation may suffer — but the formal regulatory accountability is on the certifier and the manufacturer. This is correct because the certifier was the one claiming the system makes correct domain decisions, and that's the claim that failed. The software engineer's claim was narrower — "this code does what I intended it to do" — and that claim may have been correct even when the system failed, because the intent itself was wrong in the domain.

This structure also explains why crossover engineering is more expensive than business engineering. The two-role overhead is real. Medical device companies need both software engineers and biomedical engineers. Avionics companies need both software engineers and aerospace engineers. Industrial control vendors need both software engineers and the relevant domain engineers (chemical, mechanical, electrical depending on the application). The cost of having both roles is part of the overhead that crossover work requires, and it's correctly absorbed because the True Cost magnitudes justify it.

A clarifying point about how software engineers should think about this. The software engineer in a crossover role is doing real engineering — your work is being held to the

same structural criterion as a senior civil engineer's. But the safety claim ("this system won't kill someone") doesn't rest on you alone. It rests on the certification chain that wraps around your work. Your job is to build the system rigorously, document it thoroughly, validate it carefully, surface the decision points clearly, so that the domain engineer can evaluate them and certify the result. The domain engineer's job is to make the safety claim. You're both engineers; you're doing different engineering; both pieces are needed. The structural division of labor isn't a diminishment of your role — it's the correct architecture of accountability for the activity, given the structural properties of software as a substrate.

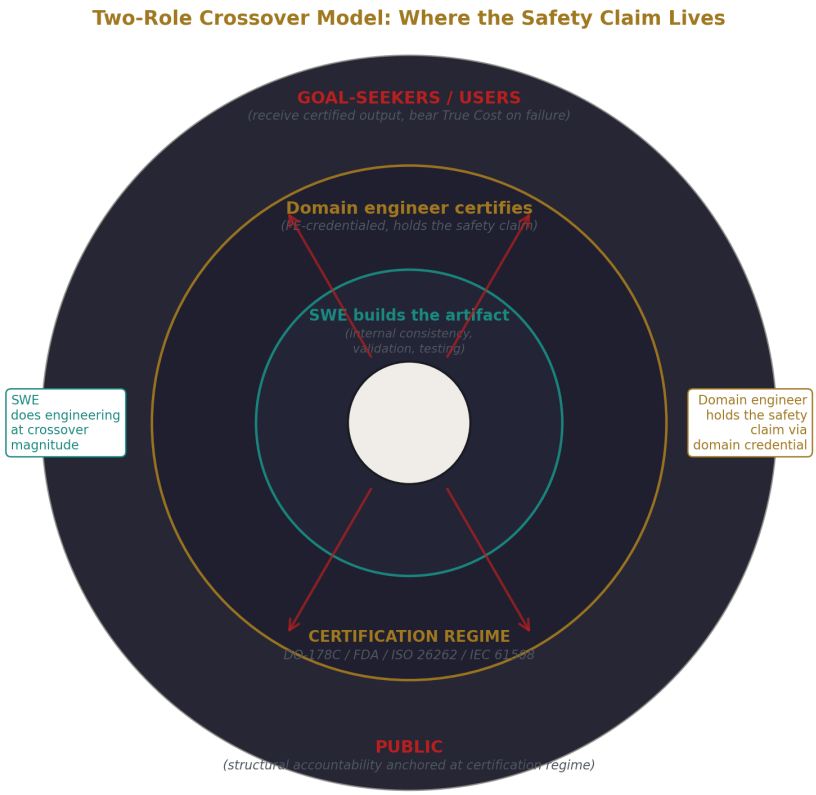

Figure 7: Fig. 7: Two-Role Model — SWE builds the system, domain engineer certifies the output. The safety claim is anchored at the certification regime, not at the SWE.

10. The complete three-tier credentialing model

The picture assembles into a clean structural answer to the credentialing question.

Traditional engineering. Cost domain: physical and civilizational. Substrate: stable. Credentialing target: the person, then the output via stamps. The chain of accountability runs through the licensed engineer. This works because the substrate holds still, competence is portable, formal codes exist, and the cost magnitude justifies the apparatus.

Business engineering. Cost domain: financial and service-availability. Substrate: unstable, rapidly evolving. Credentialing target: none. The market handles competence assessment through reputation, interviews, employment, and similar mechanisms. This is correct because the substrate moves too fast to support credentialing, the cost magnitudes don't justify the overhead, and the market handles it adequately for the costs involved. Most software engineering work is in this tier.

Crossover engineering. Cost domain: traditional (mortality, civilizational disruption) but reached via software. Substrate: the specific engineered system, which has its own internal consistency requirements. Credentialing target: the output (the system), certified by a domain-credentialed traditional engineer with software systems literacy. The software engineer who built the system is doing real engineering but is not individually credentialed for the safety claim. The domain engineer who certifies the system holds the formal accountability through their domain credential.

This is the structural answer. It handles every case the field has been confused about. It explains why general software engineering credentialing has structurally failed and will continue to fail — the substrate doesn't support it and the cost structure doesn't warrant it. It explains why the regulated software domains (avionics, medical, automotive, industrial) have evolved toward output certification by domain experts — that's the only credentialing model that fits the structural facts. It places the typical software engineer correctly in the business tier where credentialing is unnecessary and impossible. It places software engineers working on safety-critical systems correctly in the crossover tier, doing real engineering, working within a certification regime where the formal safety claim belongs to the domain engineer who certifies the output.

The same software engineer might cross between tiers across their career. Tuesday they're writing a Django web app — business tier, no credential needed. Wednesday they're writing pacemaker firmware — crossover tier, working within a domain-engineer certification regime. Thursday they're back on the Django app. The tier follows the work, not the person. The framework handles this because the activity, not the role, is the unit.

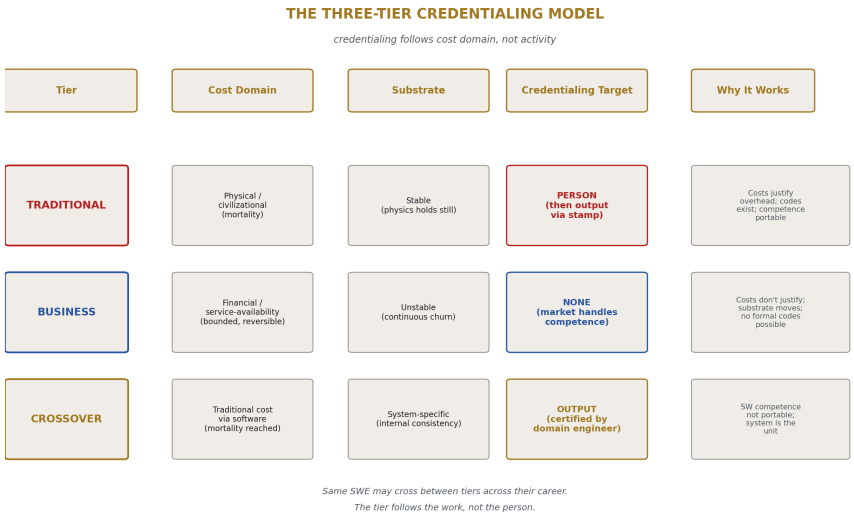

Figure 8: Fig. 3: The Three-Tier Credentialing Model — credentialing follows cost domain, not activity.

11. Why this resolves long-standing confusions

Several persistent confusions in the software industry's discourse about credentialing dissolve under this framework.

"Why don't software engineers have a PE?" Because the substrate doesn't support PE-style person-level credentialing for general software work, and the costs don't warrant it. Where software work is at PE-cost magnitudes (crossover engineering), the appropriate certification target is the system, not the person, and the person who holds the credential is a domain engineer, not a software engineer. The absence of a software PE is structurally correct. The presence of weak software certifications that try to imitate the PE is the failure mode that results from misunderstanding why the PE works.

"Should software engineers have professional ethics codes like other engineers?" The codes that matter for software work are the codes that follow True Cost. For most software work (business tier), professional ethics is bounded by employment, contract, and reputation, and society has not built additional ethics infrastructure because the cost magnitudes don't warrant it. For crossover work, professional ethics flows through the certification regime — DO-178C, FDA, ISO 26262 all carry ethical weight that licensed practitioners in those regimes must observe. Software engineers don't need a separate professional ethics code; they need to operate within the existing domain-specific ethics frameworks when their work crosses into those domains, and accept that for most of their work, normal professional and contractual ethics suffice.

"Why have IEEE certifications and ABET-accredited software engineering pro-

grams failed to achieve PE-equivalent status?" Because they're trying to credential something that can't be person-credentialed at the level of rigor a PE represents. The substrate moves too fast, competence isn't portable, codes don't exist in the formal sense traditional engineering has them. The credentialing bodies are not failing through incompetence; they're attempting a structurally impossible task. The credentials they produce are weak because what they're trying to certify isn't the right unit.

"Should self-driving car software engineers be licensed?" The right answer is: the system should be certified, not the engineers. The certifier should be a domain-credentialed traditional engineer (likely an automotive functional-safety engineer, a transportation engineer, a vehicle dynamics specialist, or a hybrid of these) with adequate software systems literacy to evaluate the artifact in context. The software engineers who build the system are doing real engineering and should be held to high standards, but the formal safety claim about the output belongs to the domain certifier, not to them individually. This is the model that ISO 26262 already implements; it should be tightened and made more rigorous as autonomous vehicles take on more decision authority, but the structural model is correct.

"Should AI safety engineers be licensed as the field grows?" The same logic applies. Where AI systems make decisions at traditional-tier cost magnitudes, the systems should be certified by domain-credentialed engineers in the relevant domains (medical AI by physicians and biomedical engineers, autonomous-systems AI by the relevant domain engineers, etc.). The AI software engineers who build the systems are doing real engineering at crossover magnitudes when their work touches these domains, but they are not the right party to hold the safety claim. The claim belongs to the domain engineer who can evaluate whether the system's behavior is correct in their substrate.

"Why do tech-industry compensation systems pay 'software engineers' so well if most of them aren't doing real engineering?" Because the market is paying for what software engineers actually produce, which is business engineering and crossover engineering value, not because the title accurately describes what most of them do. The compensation reflects the financial value the work creates and protects, which is real even when the work is in the business tier and not engineering in the strict sense. The title carries borrowed prestige from traditional engineering, but the compensation is calibrated to market value, not to title accuracy. This is fine. The title is misleading; the compensation is appropriate to the work; both can be true simultaneously.

12. What this means for software engineers

Practical consequences for individuals doing the work.

If you are a software engineer doing typical web, mobile, backend, or application work, your work is in the business engineering tier. You may or may not be doing engineering in the strict structural sense — that depends on whether you're evaluating live trade-offs against substrates that push back, with True Cost on parties outside your work. Often you are. Sometimes you're closer to skilled trades work, executing patterns in forgiving environments. Either way, no credentialing is needed and none is meaningfully available. The market handles competence assessment for your work through the mechanisms it has

— interviews, performance reviews, employment, reputation. You don't need a license. You're not failing the profession by not having one. There is nothing to license.

If you are a software engineer doing crossover work — automotive, medical, aerospace, industrial control, grid management, anything where your software's failure can kill people or disrupt civilization — you are doing real engineering at traditional-tier cost magnitudes. Your work is held to the structural criterion. Your code, when it ships, will participate in a certification regime where a domain-credentialed engineer signs off on the system's correctness in their domain. You are not the person making the safety claim, and that's structurally correct because the safety claim is too big for any one software engineer to make on their own authority — it depends on domain knowledge you don't have at the level the certifier has. Your job is to build the system rigorously enough that the certifier can do their job. Your professional accountability flows partly through the certification process and partly through your own employment and professional standing. You are an engineer in the strict sense. You also do not personally hold the safety claim. Both are true.

If you are considering whether software engineering as a profession should pursue PE-style credentialing or a similar formal apparatus: this paper argues that it shouldn't and structurally can't for general software work. The energy that has been spent on these efforts could be better spent on tightening the certification regimes that already exist for crossover work, on improving the software systems literacy of domain engineers who certify, and on developing better structural understanding of where software does and doesn't fit in the broader engineering landscape.

If you are working on software that is moving into traditional cost domains — building an AI system that will make medical diagnoses, working on software that will control critical infrastructure, writing autonomous-system decision logic — recognize that you are entering crossover territory. Your work needs domain-engineer certification. If your project doesn't have that certification structure in place, you are working without the accountability anchor that should exist for work at this cost magnitude. This is a structural problem with the project, not with you, but it is also a problem you should advocate to fix, because the absence of certification doesn't make the True Cost smaller — it just means the cost is being absorbed by users without the protective infrastructure society normally builds around traditional-tier engineering.

13. Closing

The first paper defined engineering. This paper defines what credentialing structures society should build around engineering, and what structures cannot exist regardless of effort. The two together produce a framework that addresses the major confusions in how the software industry talks about its own professional identity.

The unifying principle is that credentialing follows cost domain, not activity. Where the cost is physical or civilizational, society credentials the person and the output. Where the cost is financial and service-availability, society credentials nothing and the market does its work. Where software produces traditional-tier costs, society certifies the system and the certifier is a domain-credentialed engineer.

Three tiers. Two credentialing models. One clean structural answer that handles general software engineering, safety-critical software engineering, the relationship between software and traditional engineering disciplines, the failure of past software credentialing efforts, and the appropriate response to software's expanding role in mortality-bearing systems.

The position is defensible against pushback from regulators, licensing bodies, software professional organizations, and traditional engineering disciplines. Each constituency gets what they should get and nothing more. Traditional engineers retain their credentialing structure, which fits their work. Software engineers in the business tier are released from the unproductive search for a PE-equivalent that can't exist for their work. Software engineers in the crossover tier are placed correctly within certification regimes already built around domain expertise. Domain engineers are recognized as the appropriate certifiers for crossover work, with the software engineers they work alongside recognized as engineers in their own right with a different scope of accountability.

The paper does not argue that current arrangements are correct in every detail. Existing crossover certification regimes have gaps. Domain engineer training in software systems literacy is uneven. The boundary between business and crossover engineering is not always cleanly visible at project intake. But the structural framework is correct, and where current arrangements deviate from it, the deviations can be diagnosed and corrected. The framework is a tool for that diagnosis.

The deepest thing the framework establishes is that software engineering is not a single profession with a single credentialing question. It is a category that spans two cost domains and three engineering tiers, with different appropriate responses in each. Treating it as a single profession demanding a single credentialing answer is what produced fifty years of confusion. The confusion ends when the field is split correctly. Most software work needs no credential. Some software work needs system-level certification by domain experts. The traditional disciplines, which have already worked out the credentialing model that fits their cost domain, can keep their model. Software does not need to imitate them. Software needs to recognize where it sits in the landscape and let credentialing follow the cost.

That is the structural answer. The remainder of the work is for the field to absorb it.

[**HOWL-ENG-2-2026**] What True Cost Is. Github: https://github.com/ghowland/papers/tree/main/papers/ENG/HOWL-ENG-2-2026

[**HOWL-ENG-1-2026**] What Engineering Is. Github: https://github.com/ghowland/papers/tree/main/papers/ENG/HOWL-ENG-1-2026